1250. Check If It Is a Good Array

Hard | Topics | Companies | Hint

Given an array `nums` of positive integers. Your task is to select some subset of `nums`, multiply each element by an integer and add all these numbers. The array is said to be **good** if you can obtain a sum of `1` from the array by any possible subset and multiplicand.

Return `True` if the array is **good** otherwise return `False`.

Example 1:

```
Input: nums = [12,5,7,23]
Output: true
Explanation: Pick numbers 5 and 7.
5*3 + 7*(-2) = 1
```

Example 2:

```
Input: nums = [29,6,10]
Output: true
Explanation: Pick numbers 29, 6 and 10.
29*1 + 6*(-3) + 10*(-1) = 1
```

Example 3:

```
Input: nums = [3,6]
Output: false
```

Constraints:

- `1 <= nums.length <= 10^5`
- `1 <= nums[i] <= 10^9`

DS / Day-94 / 10-11-26

leetcode-1250 - check if it is a good array.

You're given an array nums containing +ve integers.
You need to determine whether you can:
(1) select some elements from the array (a subset).
(2) multiply each selected element by an int.
(3) add all the resulting values
(4) obtain exactly 1.

if it is possible, return true, otherwise return false.

* the integers used for multiplication can be +ve, -ve or zero.

for ex: nums = [12, 5, 7, 23]
we can select 5 & 7.
multiply them by integers,
5 x 3 = 15.
7 x -2 = -14.
add them = 1.
We obtained 1, so the array is a Good Array.

Answer = true.

* the numbers 3 & -2 are not randomly selected, they are coefficients that we calculate to make the equation equal to 1. We want to find two integers x & y such that:

$$5x + 7y = 1$$

S1 → We the euclidean algorithm, calculate GCD:-

$$7 = 5(1) + 2$$
$$5 = 2 \times 2 + 1$$
$$2 = 2 \times 1 + 0$$

Therefore,

$$\gcd(5,7) = 1.$$

S2 → work backwards,

From the 2nd equation:- $5 = 2(2) + 1$

rearrange:- $1 = 5 - 2(2)$

From the 1st equation: $2 = 7 - 5(1)$.

substitute this into our previous eqn:-

$$1 = 5 - 2(7 - 5)$$

expand:- $1 = 5 - 2(7) + 2(5)$

$$1 = 3(5) - 2(7)$$

Therefore,

$$1 = 5(3) + 7(-2)$$

✱ one imp. thing

we didn't need to find these coefficients in our actual leetcode code. why?

↳ bcuz bezout's identity already guarantees that these coefficients exist whenever the GCD is 1. our code only checks whether the GCD is 1.

think of it this way:-

- euclidean algo helps us find the GCD.
- extended " " can help us find the coefficients (like 3 & -2)
- bezout's identity explains why those coefficients exist.

↳ that's the mathematical beauty of the problem.

see we can use 12 & 23 too, but choosed 5 & 7 cuz they, simply bcuz they make the example easy to demonstrate, there is no restriction on which subset we select.

we could also use all 4 numbers or another suitable subset.

↳ we only need, gcd(12, 5, 7, 23) = 1, this guarantees that a suitable combination exists.

the gcd of the entire array being 1 is sufficient, we don't need to manually discover which elements produce 1.

Ex-2 nums = [3, 6]

gcd(3, 6) = 3

the gcd is 3 neither than 1, obtaining 1 is impossible,

therefore; Returning false not a good answer.

Reason → the GCD divides every element, so it must also divide every integer linear combination of those elements.

If the gcd is greater than 1, that combination cannot equal 1.

Bezout's identity → it states, for any two int., a & b, there exist integers x & y such that:- $ax + by = \gcd(a, b)$

let's understand this,

Suppose, a = 5, b = 7,

their GCD is, gcd(5, 7) = 1.

Bezout's identity tells us that there exists int: x & y satisfying:- $5x + 7y = 1$,

now consider, a = 6, b = 9, their GCD is, gcd(6, 9) = 3

we can write, 6(2) + 9(-1) = 3,

we can create the GCD using int. multiplication & add".

this is the key observation behind today's problem.

GCD of the entire array :

suppose, nums = [12, 5, 7, 23]

gcd (12, 5, 7, 23) = 1.

↳ According to the generalized Bezout's identity, if the gcd of a collection of int. is 1, we can find int. coefficients whose linear combination equals 1.

therefore the array is good.

Approach - 1, using the built-in GCD function

C++ provides a built-in GCD function through the <numeric> header,

#include <numeric>

the function is :- gcd (a, b).

but our array contains multiple elements. so, we can calculate it iteratively.

```
class Solution { public:
    bool isGoodArray(vector<int>& nums) {
        int g = 0;
        for (int num : nums) {
            g = gcd(g, num);
            if (g == 1) {
                return true;
            }
        }
        return g == 1;
    }
};
```

huh, gcd(0, x) = x, therefore starting with 0 allows us to process the 1st element without any special conditions.

→ initialize the gcd with 0,
→ traverse the array.
→ update the gcd.
→ we calculate the GCD of the previously calculated gcd & the current element,
if g is 1, condition is met.
directly returns a boolean depending on whether the GCD equals 1.

dry run

nums = [12, 5, 7, 23]

initially, g = 0;

iter=1 current number = 12,
g = gcd(0, 12) = 12.

iter=2 current num = 5,
g = gcd(12, 5) = 1.

iter=3 current num = 7,
g = gcd(1, 7) = 1

iter=4 current num = 23
g = gcd(1, 23) = 1.

n = no. of elements.
M = max. magnitude of the elements.
T.C = O(n log M) approx.
→ we visit every element once, O(n)
→ each gcd calculation using the euclidean algo takes logarithmic time on the size of its arguments,
→ therefore overall it is O(n log m).

S.C = O(1)

Approach 2 - Implementing the Euclidean algo manually.

now let's see what happens internally when we call: gcd(a, b),
c++ uses the euclidean algo to calculate gcd efficiently,
the mathematical rule is:-
gcd(a, b) = gcd(b, a mod b),
we repeatedly replace the larger number with a smaller one until the remainder becomes zero.
when the remainder becomes zero, the other number is the GCD.

ex: find GCD of 12 & 5.

initially: a=12, b=5,

S1 12 mod 5 = 2

replace, a=5, b=2,

S2 5 mod 2 = 1

replace, a=2, b=1,

S3: 2 mod 1 = 0

replace a=1, b=0,

now b is zero,

therefore, gcd(12,5)=1.

```
class Solution{
public:
int findGCD(int a, int b)
    {  continue until the 2nd no. becomes zero.
while(b!=0){
calculate int rem = a%b;
    a=b; replace
    b=rem;
}
return a; return the gcd.
}
```

```
bool isGoodArray(vector<int>& nums){
    int g=0;
for(int num: nums){
    g= findGCD(g, num);
}
return g==1; if the final GCD equals 1, the array is good.
}
```

calculate the gcd of the accumulated result & the current no.

approach 1

g=gcd(g, num)

built-in C++ function,

↓

C++ calculates gcd internally.

approach 2

g= findGCD(g, num)

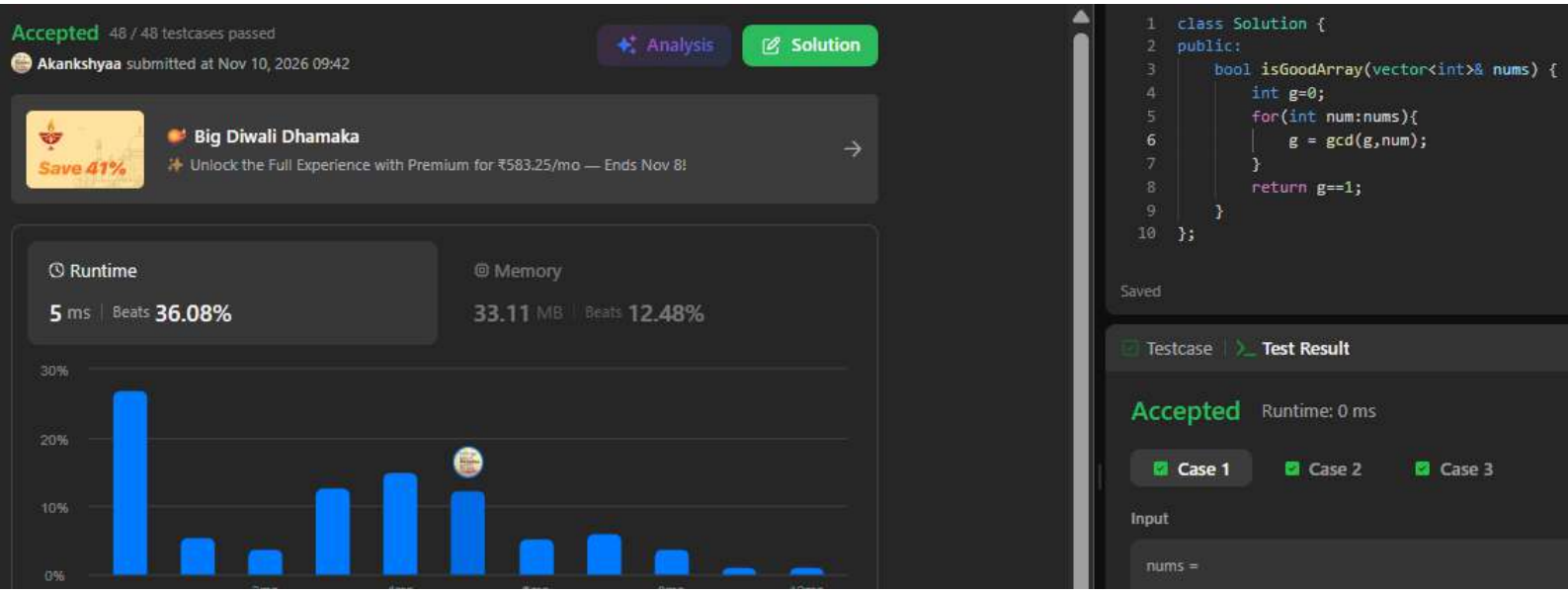
our own function

↓

we implement the euclidean algo ourselves.

T.C = O(nlogM) → gcd calculation for every element.
S.C = O(1) → only a constant no. of variables.

Why 1st approach is inefficient?
for an array of n elements, the no. of possible subsets is:- 2^n
the constraints allow:- $n \leq 10^5$
even generating all subset is impossible for such a large input
and there is another problem; the multipliers can be any integers, so trying all possible multipliers is not a finite search strategy.


Accepted 48 / 48 testcases passed
Akankshyaa submitted at Nov 10, 2026 09:42
Analysis
Solution
Save 41%
Big Diwali Dhamaka
Unlock the Full Experience with Premium for ₹583.25/mo — Ends Nov 8!
Runtime
5 ms | Beats 36.08%
Memory
33.11 MB | Beats 12.48%
30%
20%
10%
0%
1 class Solution {
2 public:
3 bool isGoodArray(vector<int>& nums) {
4 int g=0;
5 for(int num:nums){
6 g = gcd(g,num);
7 }
8 return g==1;
9 }
10 };
Saved
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =

Accepted 48 / 48 testcases passed

Akankshyaa submitted at Nov 10, 2026 10:09

Analysis Solution

Save 41%

Big Diwali Dhamaka

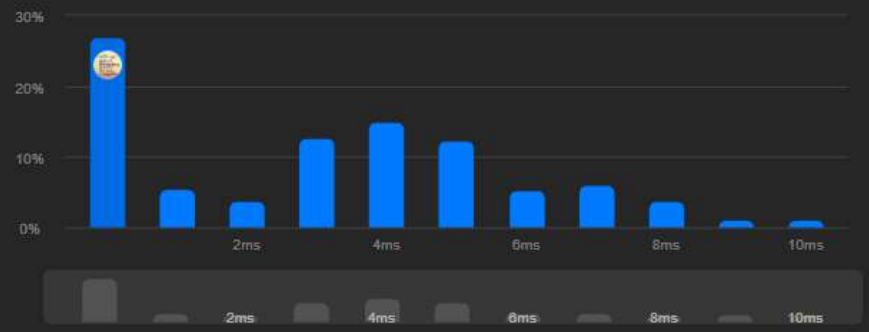
Unlock the Full Experience with Premium for ₹583.25/mo — Ends Nov 8!

Runtime

0 ms | Beats 100.00%

Memory

33.08 MB | Beats 46.64%




```cpp
class Solution {
public:
int findGCD(int a,int b){
    while(b!=0){
        int rem = a%b;
        a=b;
        b=rem;
    }
    return a;
}
    bool isGoodArray(vector<int>& nums) {
        int g=0;
        for(int num:nums){
            g = findGCD(g,num);
        }
        return g==1;
    }
};
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input